P1208R0
Adopt source_location from Library Fundamentals V3 for C++20

Published Proposal, 2018-10-07

This version:
https://rawgit.com/cor3ntin/CPPProposals/master/merge_source_location/P1208.html

Authors:
Corentin Jabot
Robert Douglas

Audience:
LEWG, LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes that `source_location` (from Library Fundamentals V3) be adopted into the C++20 standard

Table of Contents

References

Informative References

§ 1. Proposal

This paper proposes that source_location from Library Fundamentals V3 [N4758] be adopted into the C++20 standard.

`source_location` has been successfully implemented without difficulties in GCC, and Clang implementers almost completed their implementation of it. As described by the original paper, it's a necessary feature to implement logging without requiring macros, one that cannot be portably implemented by non-standard libraries because it requires compiler support.

`source_location` has been very favorably received by both EWG and LEWG [n4129], [n4417] and has been part of library fundamentals v2 [n4562] and v3 [N4758] since 2015 without changes to exposed interface.

> Note: A proposal [P0555R0] in 2017 to make use of `string_view` in `source_location` has been withdrawn by their authors once `string_view` gained a constexpr constructor, allowing the use of `source_location` in constexpr contexts.

§ 2. A few design changes

During an early review of this papers, some interest in a few design changes arose:

§ 2.1. Enforcing a size for `source_location` objects ?

`source_location` as currently implemented by Clang and GCC is a structure encapsulating all its members (file, function, line, column), and so its size is roughly `3 * sizeof(void*)`.

A note states:

> [Note: The intent of source_location is to have a small size and efficient copying. — end note]

However, there seems to be some interest for having `source_location` be guaranteed `sizeof(void*)`. Notably, reviewers wonder if `source_location` should be embedded in a yet to

be proposed `std::error`. Herb Sutter and Niall Douglas pointed out that `std::error` is meant to handle recoverable errors, rather than to communicate an error to developers (see Herb Sutter's talk on error handling CppCon 2018). And while we agree strongly with this sentiment, it's worth noting that a subset of the C committee is interested in having source information in whatever deterministic exception handling they are working towards;

While, as pointed out by Niall, it's unlikely the two languages will have directly interoperable mechanisms, having a guaranteed size for `source_location` might make such interoperability easier should a need for it arise.

Barring that, `source_location` is an error reporting tool targeted at human developers, and the cost of any such error reporting system is many orders of magnitude larger than copying 3 pointers around. We do not think there is a strong incentive to guarantee the size of that type or restricting the way it is implemented.

For completeness, such modification could be achieved by adding one level of indirection:

```
struct __source_location_data {
  /* implementation defined */
};
struct source_location {
private:
    const __source_location_data* __data;
};
```

Alternatively, source_location could return a const reference:

```
struct source_location {
    constexpr const source_location & current() noexcept;
};
```

The authors strongly prefer the first solution as we think retaining value semantics is important. It is also important to note that, while not implemented, the first solution has been considered and is deemed realistically implementable by both GCC and Clang developers (Jonathan Wakely, Eric Fiselier).

§ 2.2. Removing `current()` altogether ?

If LEWG elects to keep value semantics, the authors would like to propose that the `current` function is removed, since, in its current form, `source_location` has a default constructor that has no meaningful use. Besides being harder to misuse, using the constructor of the type to acquire its value is also significantly less verbose.

```
void log(auto && message, std::source_location sc = std::source_location::c
//or
void log(auto && message, std::source_location sc = {});
```

§ 3. Proposed Wording

Create a new header with the synopsis taken from paragraph ¶15.11 of [N4758].

Add the `source_location` class to the C++ Working paper using the content from [N4758] in the header.

Move this content from the `std::experimental::fundamentals_v3` namespace to the `std` namespace.

In section 15.5.1.3 `[library.requirements.organization.compliance]`, add the header to the table 21 (`[tab:cpp.headers.freestanding]`).

NB: The new `<source_location>` header would be best described in a new `Reflection` library (as it is in [N4758]). At the time of the publication of this category does not exist, and so this paper defers to LEWG and LWG to find the best place to describe this header in the wording or to introduce a new Refection category in the wording.

Similarly to the original [n4417] wording, we recommend the feature test macro `__cpp_lib_source_location` for this feature.

§ 4. Acknowledgments

The authors would like to thanks the people who reviewed early version of this proposal, notably Peter Dimov, Jonathan Wakely Arthur O'Dwyer and Geoffrey Romer. We would also like to thank Herb Sutter and Niall Douglas for their insightful remarks on std::error and errors handling in general.

§ References

§ Informative References

[N4129]

Robert Douglas. Source-Code Information Capture. 10 October 2014. URL: https://wg21.link/n4129

[N4417]

Robert Douglas. Source-Code Information Capture. 8 April 2015. URL: https://wg21.link/n4417

[N4562]

Geoffrey Romer. Working Draft, C++ Extensions for Library Fundamentals, Version 2. 5 November 2015. URL: https://wg21.link/n4562

[N4758]

Thomas Köppe. Working Draft, C++ Extensions for Library Fundamentals, Version 3. 25 June 2018. URL: https://wg21.link/n4758

[P0555R0]

Axel Naumann. string_view for source_location. 20170130. URL: https://wg21.link/p0555r0